Microtasks vs Macrotasks

10 min read · Updated 24 Jul 2026 · Topic 9 of 70

Why it matters

Every asynchronous callback in Node lands in one of two categories, and the category decides *when* it runs relative to everything else. **Macrotasks** are the phase-based callbacks — timers, I/O, `setImmediate`, close handlers. **Microtasks** are promise continuations and `queueMicrotask`, with `process.nextTick` as a special even-higher-priority sibling. The rule that governs their interplay is short and absolute: **after every single macrotask, the entire microtask queue is drained before the next macrotask runs.**

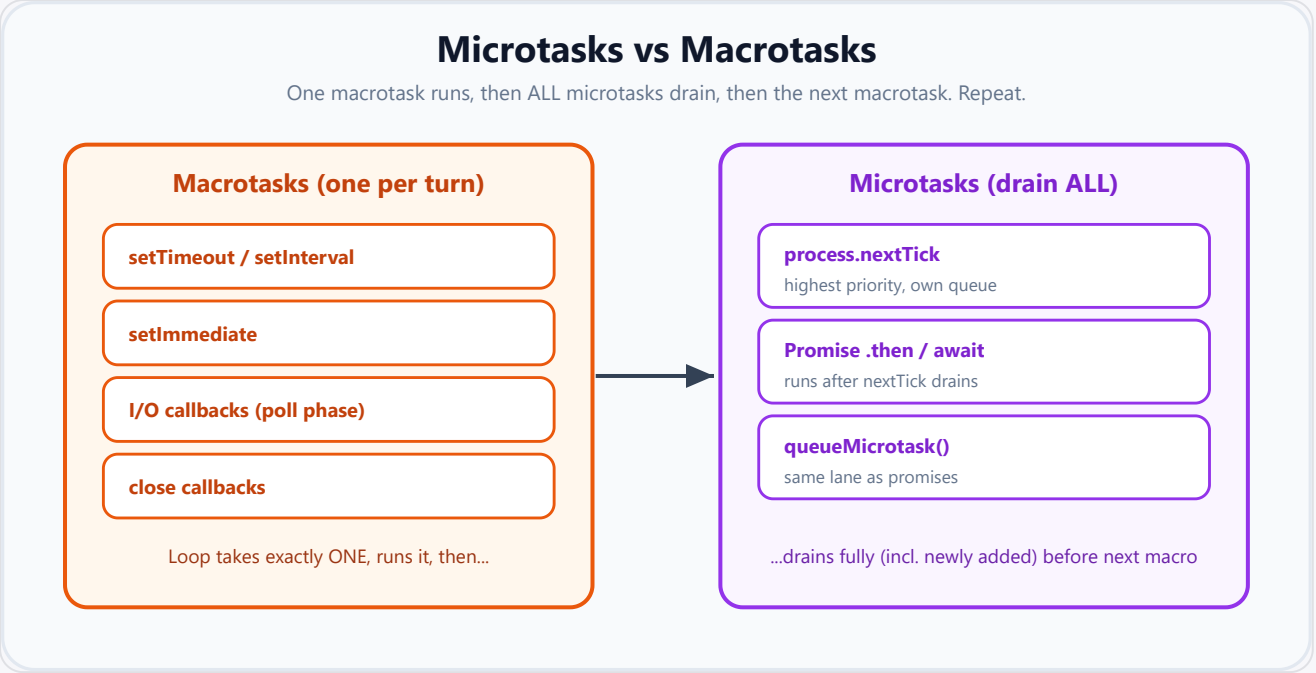
This is why `await` feels synchronous-ish but never blocks, why a promise chain always completes before a `setTimeout` that was scheduled earlier, and why a runaway promise chain can starve I/O just like a runaway `nextTick`. It is also the concept that unifies Node with the browser: the microtask/macrotask model is the same in both, which is why understanding it makes you better at *all* JavaScript, not just backend. Interviewers ask output-ordering questions precisely because they cannot be bluffed — either you can trace the queues or you cannot.

The concept, simply

Two kinds of work, two different scheduling rules:

- **Macrotask:** the loop takes *one* per turn. `setTimeout` / `setInterval` (timers phase), I/O callbacks (poll phase), `setImmediate` (check phase), and close callbacks are all macrotasks. Each sits in a phase queue and the loop processes them as it walks the phases.
- **Microtask:** the loop drains *all* of them — repeatedly, including any newly scheduled during the drain — after each macrotask and at every phase boundary. Promise

queue that drains *before* the promise microtasks.

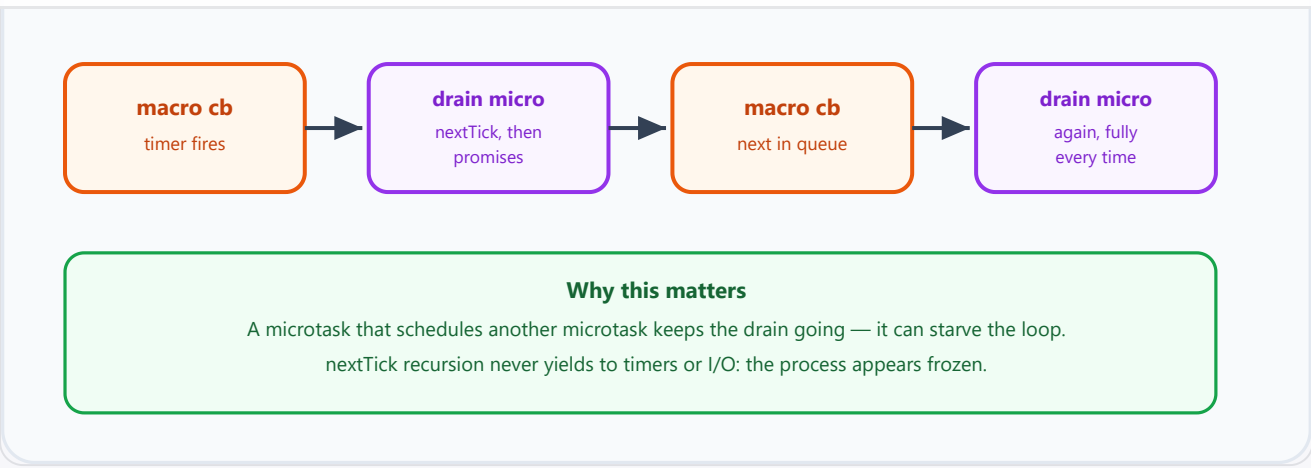


The loop takes one macrotask, then empties the whole microtask queue, then takes the next macrotask.

Mnemonic: **one macro, then all micros, then one macro, then all micros.** Microtasks are greedy; macrotasks are polite and take turns.

How it works

The drain is exhaustive. When Node drains microtasks, it does not process a fixed snapshot — it keeps going until the queue is genuinely empty. If microtask A schedules microtask B, and B schedules C, all of them run in the same drain, before the loop is allowed to move on. This is what makes `await` chains resolve in one continuous burst. It is also the starvation risk: an endless chain of promises-that-schedule-promises never lets the loop reach the next macrotask, freezing timers and I/O — exactly the `nextTick` starvation problem, one lane over.



Microtasks drain fully after each macrotask; a self-scheduling microtask chain can starve the loop just like recursive nextTick.

nextTick vs promise, precisely. Node maintains two distinct queues in the microtask family. The `process.nextTick` queue is drained first and completely; then the V8 Promise microtask queue is drained. If a nextTick callback schedules a promise, and a promise callback schedules a nextTick, Node re-checks: after promises run, if new nextTicks appeared, it drains those too, then re-drains promises — it keeps alternating until both are empty. The net effect you can rely on: **at the same scheduling point, nextTick beats promise.**

How `await` maps onto this. An `await x` is sugar for "evaluate `x`, then schedule the rest of this function as a promise microtask". Everything before the first `await` runs synchronously; the moment you hit `await`, the function suspends and control returns to the caller. The continuation resumes as a microtask once the awaited value settles. So an `async` function is partly synchronous (up to the first `await`) and partly a series of microtasks (each resumption after an `await`).

Where macrotasks fit around all this. The loop enters a phase, runs one macrotask callback, then drains nextTick + promises fully, then runs the next macrotask callback in that phase, drains again, and so on until the phase queue empties — then advances. So microtasks are not "once per loop" or "once per phase"; they are *once per macrotask callback*. That granularity is the crux of most trick questions.

The two families at a glance. The table fixes the distinction that every ordering puzzle turns on — *how many* the loop takes per turn, and *when* it comes back for more.

Examples	`setTimeout`, `setInterval`, `setImmediate`, I/O, close callbacks	`Promise.t`
Lives in	A phase queue (timers, poll, check, close)	The nextTic
How many run per turn	Exactly one callback, then the loop pauses to drain microtasks	The entire
When it next runs	Next time the loop reaches that phase	After every
Starvation risk	Low — always yields between callbacks	High — a se

A concrete trace that exposes the "once per macrotask" rule. Suppose two timers are both due, and each timer callback schedules a promise. The loop enters the timers phase, runs timer A, and *before touching timer B* it fully drains the microtask queue — so A's promise runs. Only then does it run timer B, and again drains microtasks, so B's promise runs. The output is `A, A-promise, B, B-promise`, never `A, B, A-promise, B-promise`. If microtasks drained "once per phase" instead, you would see the second, wrong ordering. Reproducing this by hand is the single best way to prove to yourself that the drain happens between the two macrotasks, not after both.

Code examples

Example 1 — micro always beats macro.

JAVASCRIPT Copy

```
console.log('sync 1');
setTimeout(() => console.log('macro: timeout'), 0);
Promise.resolve().then(() => console.log('micro: promise'));
console.log('sync 2');

// Output:
```

```
//   sync 1
//   micro: promise    ← microtask drains before the loop reaches timers
//   macro: timeout
```

Example 2 — a chain of promises drains before any timer.

JAVASCRIPT Copy

```javascript
setTimeout(() => console.log('timeout'), 0);

Promise.resolve()
  .then(() => console.log('p1'))
  .then(() => console.log('p2'))
  .then(() => console.log('p3'));

// Output:
//   p1
//   p2
//   p3
//   timeout
```

Each `.then` schedules the next as a microtask; the whole chain empties in one drain before the timers phase gets its turn.

Example 3 — async/await ordering.

JAVASCRIPT Copy

```javascript
async function main() {
  console.log('A');
  await null;          // suspends here; rest becomes a microtask
  console.log('C');
}

console.log('start');
main();
console.log('end');

// Output:
//   start
//   A         ← runs synchronously up to the await
```

Example 4 — the full six-way ordering trap.

JAVASCRIPT Copy

```
console.log('1: sync');
setTimeout(() => console.log('2: setTimeout'), 0);
setImmediate(() => console.log('3: setImmediate'));
Promise.resolve().then(() => console.log('4: promise'));
process.nextTick(() => console.log('5: nextTick'));
(async () => { await null; console.log('6: await'); })();
console.log('7: sync');

// Output:
//   1: sync
//   7: sync
//   5: nextTick        ← nextTick queue drains first
//   4: promise         ← then the promise microtask queue ...
//   6: await           ← ... including the await continuation (FIFO)
//   2: setTimeout / 3: setImmediate  (macrotasks; order races at top leve
```

Sync code (1, 7) first. Then all microtask-family work: nextTick (5) ahead of promises, then the promise queue in FIFO order (4 before 6, because `.then` was scheduled before the `await` continuation). Finally the two macrotasks, whose order is a top-level race.

Common mistakes & gotchas

- **Thinking microtasks run "once per loop iteration".** They drain after *every* macrotask callback and at every phase boundary — far more often.
- **Assuming `setTimeout(fn, 0)` beats a promise scheduled just before it.** It never does; the microtask queue empties before the loop reaches the timers phase.
- **Believing `await` blocks.** It suspends the async function and returns control to the caller; the rest resumes later as a microtask.

- **Creating an unbounded microtask chain** (a promise that always schedules another) — it starves timers and I/O exactly like recursive `nextTick`.
- **Confusing FIFO within a queue with priority across queues.** Within the promise queue it is FIFO; but nextTick as a queue outranks the whole promise queue.
- **Forgetting synchronous code beats everything.** Nothing async runs until the current call stack is empty.
- **Assuming microtasks drain "once per phase".** They drain after *each macrotask callback* — so two due timers interleave with their own microtasks (`A, A-promise, B, B-promise`), not `A, B, A-promise, B-promise`.
- **Expecting a single big microtask drain to be interruptible by I/O.** It is not — until the queue is empty the loop cannot reach the poll phase, so a long promise chain delays every pending socket read just like blocking code would.

Best practices

- **Prefer `queueMicrotask` or promises over `process.nextTick`** in application code unless you specifically need to beat the promise queue.
- **For CPU-bound chunking, use `setImmediate` (a macrotask), not microtasks,** so I/O gets serviced between chunks.
- **Do not throw inside a microtask without handling it** — an unhandled promise rejection propagates differently and can crash the process on modern Node defaults.
- **Trace ordering by drawing the two queues** when a bug depends on it, rather than adding `setTimeout(…, 100)` "to be safe".
- **Keep microtask work small.** Long synchronous work inside a `.then` still blocks the loop just as much as anywhere else.

⭐ Interview lens

> **Q: Define microtask vs macrotask in Node.**
> A macrotask is a phase-based callback — `setTimeout`, I/O, `setImmediate`, close handlers — and the loop runs one per turn. A microtask is a promise continuation,

Q: Output? `setTimeout(() => console.log('t'), 0); Promise.resolve().then(() => console.log('p'));`

`p` then `t`. The microtask queue empties before the loop advances to the timers phase, so the promise wins regardless of scheduling order.

Q: How often do microtasks drain — once per loop, once per phase, or more?

More: after *every* macrotask callback, plus at each phase transition. That is why a promise chain fully resolves before the next timer, and why a self-scheduling microtask can starve the loop.

Q: Trace this async snippet. `console.log(1); (async()=>{console.log(2); await null; console.log(4);})(); console.log(3);`

Output: 1, 2, 3, 4. Code runs synchronously to the `await` (1, 2), control returns to the caller (3), then the continuation resumes as a microtask (4).

Q: Between a `process.nextTick` and a `Promise.then` scheduled together, which runs first, and why?

The nextTick callback. Node drains the dedicated nextTick queue completely before draining the Promise microtask queue, so nextTick always wins at the same scheduling point.

Q: Can microtasks starve the event loop like `nextTick` can?

Yes. Because the microtask queue is drained exhaustively — including newly added microtasks — an endless chain of promises-that-schedule-promises never yields to the loop, freezing timers and I/O. The fix is to move periodic work onto `setImmediate`.

- **Rule:** one macrotask → drain ALL microtasks → next macrotask.
- **Macrotasks:** `setTimeout`, `setInterval`, `setImmediate`, I/O, close.
- **Microtasks:** promise `.then`/`await`, `queueMicrotask`; `nextTick` = higher-priority sibling.
- Priority order: sync → nextTick → promises → macrotasks.
- Microtasks drain after every macrotask + every phase boundary (not once per loop).
- `await` = suspend + resume-as-microtask; never blocks.
- Endless microtask chains starve the loop → use `setImmediate` to yield.

QUICK CHECK — no marks, just for you.

1. What is the governing rule between macrotasks and microtasks?

- ◯ All macrotasks run, then all microtasks
- ◯ After every single macrotask, the entire microtask queue is drained before the next macrotask
- ◯ They alternate one-for-one
- ◯ Microtasks run only once per full loop iteration

Check answer

2. Which of these is a macrotask rather than a microtask?

- ◯ `Promise.then`
- ◯ The continuation after an `await`
- ◯ `setTimeout`

Check answer

3. Can a self-scheduling chain of promises starve the event loop the way recursive `nextTick` can?

- ○ No, promises can never starve the loop
- ○ Only if the promises reject
- ○ Only in the browser, not in Node
- ○ Yes — the microtask queue is drained exhaustively, so an endless promise chain never lets the loop advance

Check answer

‹ Previous
process.nextTick vs setImmediate

Next ›
Blocking the Event Loop

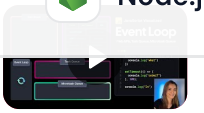
LEARN MORE

ARTICLES & DOCS

- Using microtasks in JavaScript with queueMicrotask()
  developer.mozilla.org
- In depth: Microtasks and the JavaScript runtime environment
  developer.mozilla.org
- Understanding setImmediate() (nextTick / microtask / macrotask queue order)
  nodejs.org

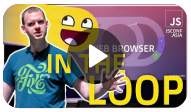
VIDEOS

Node.js Backend Kit



JavaScript Visualized - Event Loop, Web APIs, (Micro)task Queue

Lydia Hallie



Jake Archibald: In The Loop — event loop, setTimeout, micro tasks (JSConf.Asia)

JSConf

Hand-picked references for this lesson.